

Profile Service Microservice

A RESTful microservice for managing user profiles, with authentication and containerised deployment

Overview

This project is based on a university report where I designed, built and tested a profile microservice. The service manages user profile data for a community app. It is written in Python, follows RESTful API design, stores data in Microsoft SQL Server, and runs in a Docker container. Authentication is handled by a separate service, and the API is documented with Swagger.

Disclaimer

For this portfolio version I changed the app name, the entity names, the sample data and the server details. Margins is a fictional app. The design, the code and the testing approach are the ones I used in my coursework.

What the Service Does

The ProfileService manages profiles for Margins, a fictional reading community app. A normal user can create their own profile, view it, update it and delete it. An admin can view, update or delete any user's profile, for example to deal with a support ticket or remove a spam account. Every action is checked against an external authenticator before it runs.

Design

Use Case Diagram

The actors are a User, an Admin and the external Authenticator. Before any action runs, the user is checked by the authenticator. Normal users can only act on their own profile. Admins have extra actions that work on any profile.

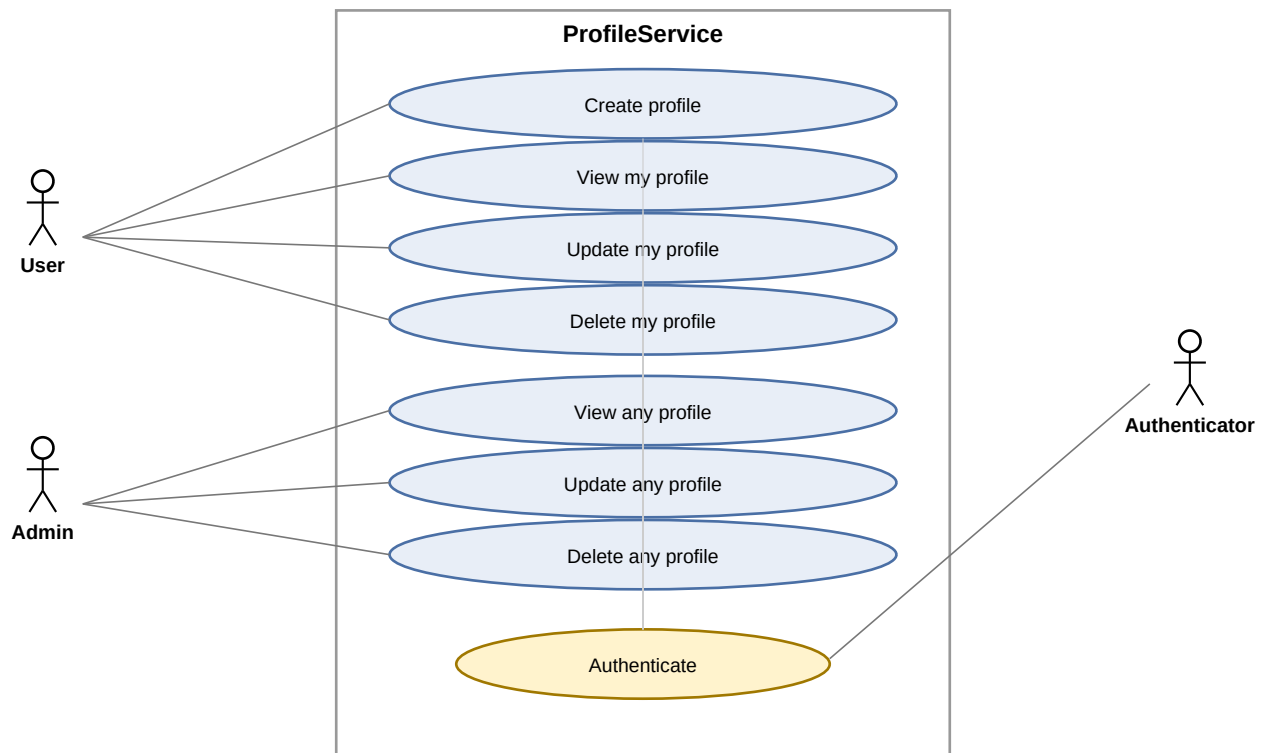


Figure 1. Use case diagram for the ProfileService

Database Design

The service stores profiles, the genres each user likes, and the city they are based in. USERGENRE is the linking table that handles the many to many link between users and genres. A log table records when a new user is added.

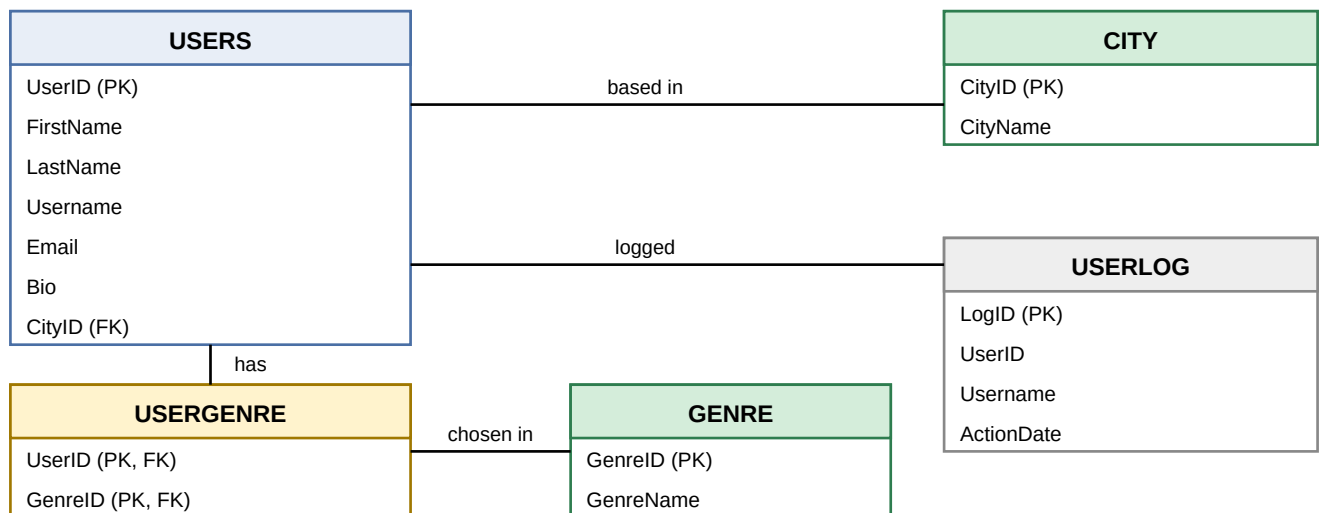


Figure 2. Entity relationship diagram for the profile database

Security and Data Handling

The service handles personal data, so security and privacy shaped the design. The main choices are below.

Delegated Authentication

The service does not store or check passwords itself. It relies on a separate authenticator service to confirm who the user is. This means the profile database holds no login details, which lowers the risk if the profile data is ever exposed. Each request is checked for a valid token before any protected action runs.

```
def require_jwt(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        auth_header = request.headers.get("authorization")
        if not auth_header or not auth_header.startswith("Bearer "):
            abort(401, "invalid token")

        token = auth_header.split(" ")[1]
        try:
            decoded = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        except jwt.ExpiredSignatureError:
            abort(401, "token expired")
        except Exception:
            abort(401, "invalid token")

        request.user_email = decoded["email"]
        request.user_role = decoded["role"]
        return func(*args, **kwargs)
    return wrapper
```

Role Based Access Control

Normal users can only reach their own profile. Before returning a profile, the service checks that the logged in user matches the profile being asked for.

```
if request.user_email != email:
    abort(403, "cannot access another user's profile")
```

Admin only actions are kept in a separate file and guarded by their own check, so it is clear which endpoints need higher privileges.

```
def require_admin(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        role = getattr(request, "user_role", "User")
        if role is None or role.strip().lower() != "admin":
            abort(403, "admins only")
        return func(*args, **kwargs)
    return wrapper
```

Keeping Secrets Out of the Code

Database credentials and other secrets are read from environment variables, not written into the source code. This stops them from being exposed if the code is shared.

```
db_server    = os.getenv("db_server")
db_name      = os.getenv("db_name")
db_username  = os.getenv("db_username")
db_password  = os.getenv("db_password")
```

Input Validation

Data models set the shape of the data going in and out. Checking incoming data against these models lowers the risk of bad or incomplete data being saved.

```
class User(db.Model):
    __tablename__ = "USERS"
    __table_args__ = {"schema": "margins"}

    UserID      = db.Column(db.Integer, primary_key=True)
    Email       = db.Column(db.String(100), nullable=False, unique=True)
    Username    = db.Column(db.String(50), nullable=False, unique=True)
    Bio         = db.Column(db.String(500))
    CityID      = db.Column(db.Integer, db.ForeignKey("margins.CITY.CityID"))
    Role        = db.Column(db.String(5), nullable=False, default="User")

    genres = db.relationship("Genre", secondary="margins.USERGENRE")
```

Implementation

Structure

The service is split into Python files, each with a clear job. The main setup links a Connexion and Flask app to a Swagger file, so the API routes are declared in YAML. SQLAlchemy sits between the app and the database. This keeps the code tidy and easy to maintain.

API and Documentation

The API is defined with Swagger (OpenAPI). The file lists every endpoint, its parameters, the security needed and the possible responses. Connexion maps each endpoint to the matching Python function, so the code always matches the documented API.

```

/users/profile/{email}:
  get:
    operationId: "users.get_my_profile"
    tags: ["Users"]
    summary: "get my profile page"
    parameters:
      - name: "email"
        in: path
        required: true
        schema:
          type: string
          format: email
    responses:
      "200": { description: "successfully returned profile" }
      "404": { description: "user not found" }

```

Deployment

The service runs in a Docker container, so it works the same on any machine. Containerisation means it can be pulled and run without manual setup, which also made testing and marking easier.

Testing

Testing checked that the endpoints behaved correctly, that security was enforced, and that the service matched the design. I used Swagger first to confirm every endpoint worked, then Postman to send real requests with and without tokens.

Example: Access Control Working

A normal user was blocked from an admin only endpoint, while an admin was allowed through. This confirmed role based access control was working.

```

# Normal user calls an admin endpoint
GET /api/admin/users/15      (token: role = User)
-> 403 Forbidden
  { "detail": "admins only", "status": 403 }

# Admin calls the same endpoint
GET /api/admin/users/15      (token: role = Admin)
-> 200 OK
  { "UserID": 15, "Username": "Leo Marsh",
    "Email": "leo@margins.example", "Role": "Admin" }

```

Functional and Security Testing

- Valid requests returned the expected JSON, and bad input returned a clear error.
- Trying to open another user's profile returned 403 Forbidden.
- Every protected endpoint rejected requests with no token.
- No personal data like credentials appeared in any API response.

The service behaved the same across all the tests, which showed it was reliable and that authentication and access control worked as intended.

Reflection and Future Work

The modular structure, the split between user and admin functions, and the use of Swagger made the service easy to test. Running it in Docker kept it consistent across machines.

- Add more detailed input validation and automated tests, instead of only manual testing in Postman.
- Expand error handling to give clearer feedback, while still hiding personal details.
- Add filtering to the admin endpoints so they scale as more users are added.

Skills Demonstrated

- RESTful API design in Python
- Token based authentication and role based access control
- Secure handling of secrets and personal data
- SQLAlchemy and Microsoft SQL Server
- API documentation with Swagger
- Docker containerisation
- Functional and security testing

Outcome

This project shows how I build a secure, documented microservice from design through to testing. It brings together database design, a REST API, token based authentication, role based access control and containerised deployment, with security and privacy considered at each step.